

Relatório de Atividades: Desenvolvimento de Microserviço com Jakarta EE

Disciplina: Sistemas Distribuídos

Instituição: UFC - Campus Quixadá

Autor(a): Gabriely Correia Dealem

Autor(a): Laura Soléria Lobo Vitorino Maciel

1. Introdução e Objetivo

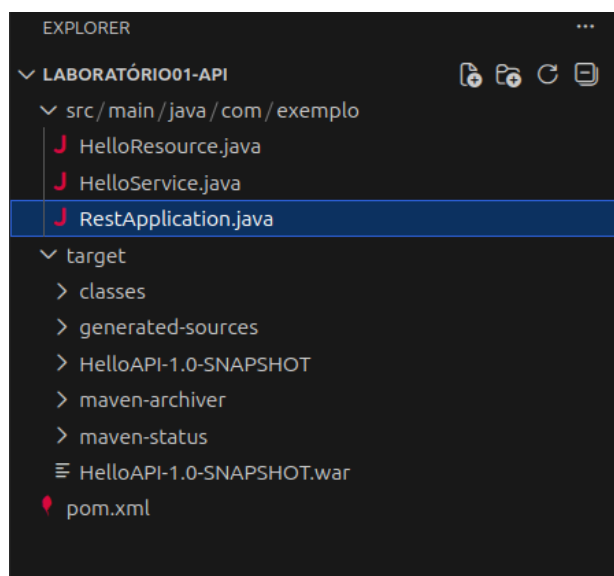
Esta atividade consistiu na criação de um serviço RESTful empregando tecnologias do **Jakarta EE 10**. O foco central foi a integração entre um recurso JAX-RS (exposição de endpoint) e um componente de lógica de negócio EJB Stateless 4. O objetivo final foi a geração de um artefato `.war` e sua execução em um servidor de aplicações corporativas.

2. Tecnologias e Ferramentas

- **Linguagem:** Java 17.
- **Servidor de Aplicação:** WildFly 34.0.0.Final.
- **Gerenciador de Build:** Maven.
- **Frameworks:** JAX-RS para REST e Jakarta Enterprise Beans (EJB).

3. Arquitetura e Estrutura do Projeto

A aplicação foi organizada seguindo os padrões recomendados para projetos Maven e Jakarta EE, garantindo a separação de responsabilidades:

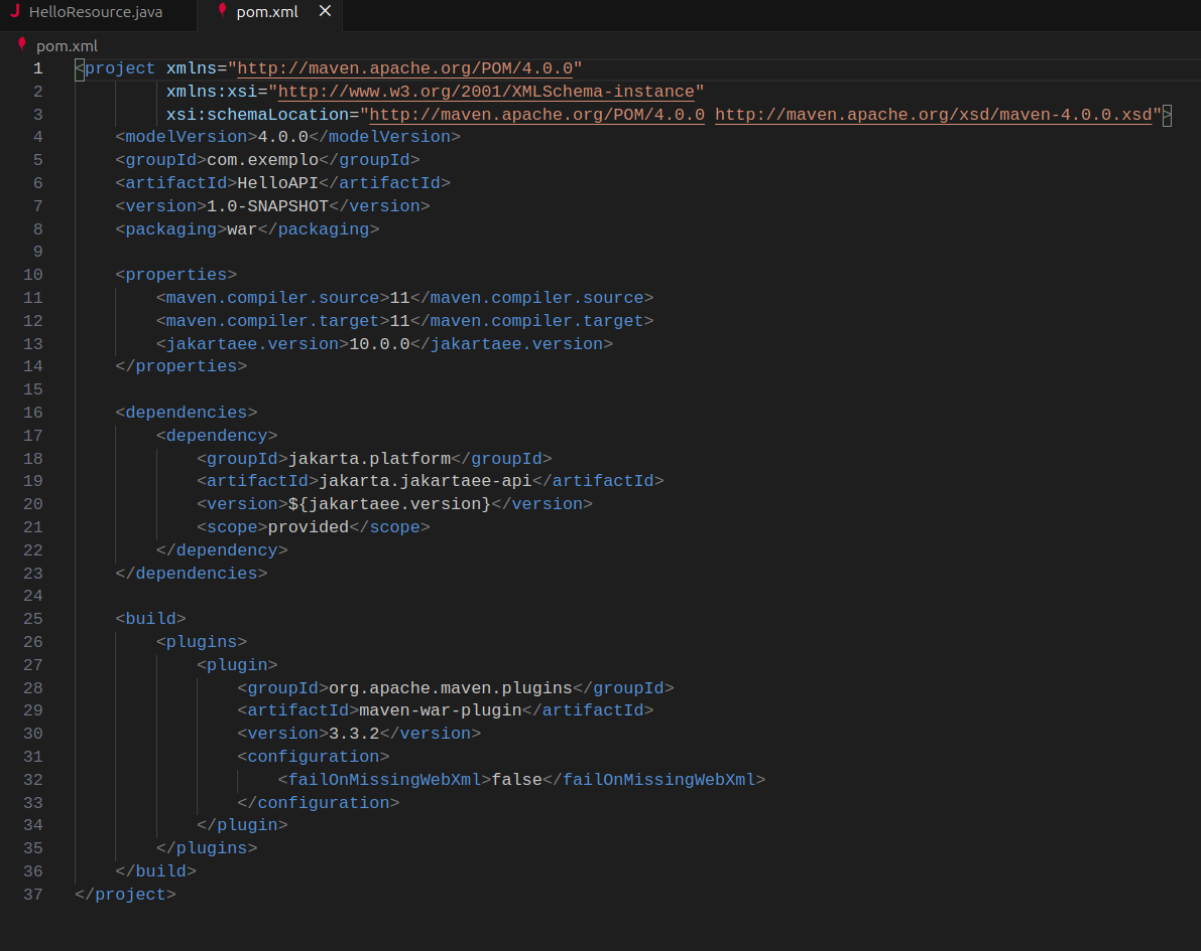


4. Implementação da Arquitetura

O projeto foi segmentado em três pilares principais:

4.1. Configuração do Maven (pom.xml)

O arquivo `pom.xml` foi configurado para gerenciar as dependências do Jakarta EE 10 e garantir que o projeto fosse compilado corretamente para a versão 17 do Java.



```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0"
3         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
5     <modelVersion>4.0.0</modelVersion>
6     <groupId>com.exemplo</groupId>
7     <artifactId>HelloAPI</artifactId>
8     <version>1.0-SNAPSHOT</version>
9     <packaging>war</packaging>
10
11     <properties>
12         <maven.compiler.source>11</maven.compiler.source>
13         <maven.compiler.target>11</maven.compiler.target>
14         <jakartaee.version>10.0.0</jakartaee.version>
15     </properties>
16
17     <dependencies>
18         <dependency>
19             <groupId>jakarta.platform</groupId>
20             <artifactId>jakarta.jakartaee-api</artifactId>
21             <version>${jakartaee.version}</version>
22             <scope>provided</scope>
23         </dependency>
24     </dependencies>
25
26     <build>
27         <plugins>
28             <plugin>
29                 <groupId>org.apache.maven.plugins</groupId>
30                 <artifactId>maven-war-plugin</artifactId>
31                 <version>3.3.2</version>
32                 <configuration>
33                     <failOnMissingWebXml>>false</failOnMissingWebXml>
34                 </configuration>
35             </plugin>
36         </plugins>
37     </build>
38 </project>
```

4.2. Camada de Negócio (EJB)

A classe `HelloService` foi anotada com `@Stateless`, garantindo que o servidor gerencie instâncias eficientes para processar a saudação personalizada.

```
J HelloService.java X
src > main > java > com > exemplo > J HelloService.java
1  package com.exemplo;
2  import jakarta.ejb.Stateless;
3
4  @Stateless
5
6  public class HelloService {
7      public String sayHello(String name) {
8          return "Olá, " + name + "! Bem-vindo à API EJB.";
9      }
10 }
```

4.3.Interface de Exposição (JAX-RS)

Através da classe `HelloResource`, definiu-se o caminho do endpoint via `@Path`. Utilizou-se `@Inject` para desacoplar a chamada do serviço de negócio.

```
J HelloResource.java X
src > main > java > com > exemplo > J HelloResource.java
1  package com.exemplo;
2
3  import jakarta.inject.Inject;
4  import jakarta.ws.rs.GET;
5  import jakarta.ws.rs.Path;
6  import jakarta.ws.rs.PathParam;
7  import jakarta.ws.rs.Produces;
8  import jakarta.ws.rs.core.MediaType;
9
10 @Path("/hello")
11 public class HelloResource {
12     @Inject
13     private HelloService helloService;
14
15     @GET
16     @Path("/{name}")
17     @Produces(MediaType.TEXT_PLAIN)
18     public String sayHello(@PathParam("name") String name) {
19         return helloService.sayHello(name);
20     }
21 }
```

4.4. Configuração da Aplicação

A classe `RestApplication` mapeou o contexto base da API como `/api`.

```
J HelloResource.java  J RestApplication.java X
src > main > java > com > exemplo > J RestApplication.java
1  package com.exemplo;
2
3  import jakarta.ws.rs.ApplicationPath;
4  import jakarta.ws.rs.core.Application;
5
6  @ApplicationPath("/api")
7  public class RestApplication extends Application {
8  }
```

5. Procedimentos de Build e Deploy

A construção do pacote foi realizada através do comando `mvn clean package`, que consolidou as classes e dependências no diretório `target/`.

Superação de Desafios Técnicos

Terminal 1: Durante a execução, identificou-se uma falha crítica de inicialização relacionada ao Cgroup Metrics do Java em ambiente Linux. O problema foi mitigado configurando a variável de ambiente:

- `export_JAVA_OPTIONS="-XX:-UseContainerSupport"`
`/home/gaby/wildfly/bin/standalone.sh`

Terminal 2: Com o servidor estabilizado, a implantação foi concluída movendo o arquivo para a pasta de auto-deploy:

- `mvn clean package`
- `cp target/HelloAPI-1.0-SNAPSHOT.war ~/wildfly/standalone/deployments/`

6. Resultados e Validação

A funcionalidade foi verificada via terminal utilizando a ferramenta `curl` no terminal 2. A chamada para o endereço local retornou a string processada pelo EJB com sucesso:

Comando: `curl http://localhost:8080/HelloAPI-1.0-SNAPSHOT/api/hello/Gaby`

Retorno: Olá, Gaby! Bem-vindo à API EJB.

7. Conclusão

A prática demonstrou a eficácia do modelo de componentes Jakarta EE para o desenvolvimento distribuído. A separação entre a interface REST e o processamento via EJB proporcionou uma arquitetura limpa e escalável, validada pelo deploy bem-sucedido no ambiente WildFly.